Database Modeling and Design (CS312)

Highlighted Handouts BY VU-Gateway:

For require Imp mid + Final Term Files Objective + Subjective then Cntct on Wytsyp: +92 305 5868956

Join VU-Gateway Group:

https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

VU-Gateway

Contents

Module-01: Introduction to Data & Information 3
Module 02: Data Storage Mechanism 6
Module 03: Three-Tier Architecture 8
Module 04: Fact Finding Technique 12
Module 05: Implementing Fact Finding Techniques 16
Module 06: Process to Database Design 19
Module 07: Relational Database 26
Module 08: Conceptual Data Modeling and Entity Relationship Diagram Overview 32
Module 09: Entities, Attributes and Relationship 34
Module 10: Extended Entity Relationship Diagram 43
Module 11: Example of Entity Relationship Diagram (ERD) 46

VU-Gateway

Module-01: Introduction to Data & Information

A. Definition of Data

Data can be referred to raw material from which we can draw conclusion after analysis or the facts from which we can infer new facts. Data has nothing to do with decision making but the data lays down foundation for decision making. Data is collected and further analyzed to convert it into information suitable for decision making. Data and information are two separate things and are not to be confused with each other.

B. Example of Data

The followings are the examples of data:

- *Telephone Directory*: it's a huge collection of Data, with an internal structure of representation i.e. name, telephone number, city or address. Facts with structures are also termed as data.
- *Student Record on Computer*: a file on computer containing data of ten thousand students is also a data. This data too have internal structure in the form of Student ID, Student Name, CGPA and likes.

C. Definition of Information

Information can be defined as processed or organized data presented in a given context so as to make it useful related to the problem in hand. When a specific set of data is analyzed or interpreted, it becomes information that is more suitable for decision making. Simply stating, data becomes information when it becomes relevant to the decision problem.

D. Examples of Information

The followings are the examples of information:

- *Telephone Directory: as stated above, telephone directory is a huge set of data and when we process this set of data to get the telephone number of a specific dentist or a colleague, it becomes information.*
- *Student Record on Computer: a file on computer containing data of ten thousand students is again a huge set of data, converts in to information when processed to get list of a students with CGPA more than three.*

E. Comparison of Data & Information

Data and information are often considered to be a synonym of each other but the truth is that there is nothing in common between these two. Both have different meaning and characteristics. Information is a subset of data, typically required set of data. Data leads to information so information is considered to be dependent of data.

F. Example to elaborate difference between Data and Information

Consider the following example to understand the difference between Data and Information:

VU-Gateway

Consider a file storing flight routes from one city to another. Data is structured in the form of Flight ID, Origin, Destination, Date of Departure, Time of Departure etc. The file contains two thousand numbers of routes. This file qualifies to be called Data because it's a huge set of structured data.

This huge data will turn into information when processed or filtered to get information about flight for a specific destination, let's say from Lahore to Karachi on April 15th.

Sample Data				
Flight ID	Origin	Destination	Data of Flight	Time of Flight
PK-725	Islamabad	Lahore	7-Apr-15	8:00
PK-702	Lahore	Karachi	15-Apr-15	22:00
PK-702	Lahore	Karachi	15-Apr-15	10:00
SI-829	Karachi	Dubai	12-Apr-15	13:00
GI-390	Lahore	Istanbul	28-Apr-15	17:00
DATA				
Question: Flight Details from Lahore to Karachi on 15-April-15				
PK-702	Lahore	Karachi	15-Apr-15	22:00
PK-702	Lahore	Karachi	15-Apr-15	10:00
INFORMATION				

G. Life With& Without Database

Database is organized data with digital procedures, which is able to handle different set of data with a very large capacity. Database is a huge repository of data and it comes with ease to manage and therefore database has become a substitute for the manual handling of lots of file with very high chances and risk of data loss. Access to data has become easy and less time-consuming activity due to the databases.

As mentioned above, advantages of using databases are enormous. Benefits of using databases vary from reducing redundancy in data to getting rid of unnecessary set of data. Without databases, loads of data is to be handled manually with many potential risks and difficult to access. With databases, it's easy to access large set of data when required. Example of Driving License and Passport can serve as a good example in which the profile data is same but it's repeated over again.

I. Why Database not Information Base

Databases deals with the storage or organization of data and the main focus of databases are to optimize data storage. On the other hand, information – as it is a subset of data - is all about the retrieval of specific data for different purposes. As information is dependent on data so databases are basic requirements for information which can then be used for multiple purposes.

J. Difference between Database and Database Management System (DBMS)

Database is all about design and structure of the data and DBMS works as a tool to manipulate or analyze the data inside the databases. DBMS is a whole system to manage a digital database, its storage, creation and retrieval (information). Differentiating between these two terms, Database is a huge set of data while DBMS is a tool to manage or control that data in a database with major function of data retrieval.

© Copyright Virtual University of Pakistan

K. Example of Database and Database Management System (DBMS)

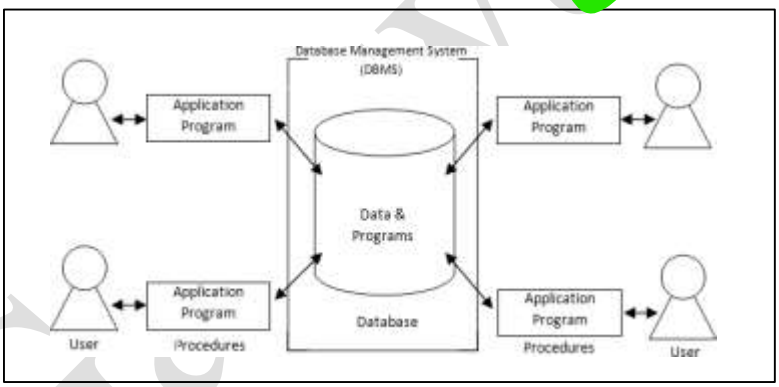
Example 01:

Database: Employee Records Column with Employee ID, Name, Qualification, Date of Joining etc.
DBMS: Oracle to create and load employee records

Example 02:
Database: Individual files on Hard Disk
DBMS: Operating System to read the file from Hard Disk

L. Components of Database Management System

Database Management System DBMS is the aggregate of the followings:



- Data inside the database which is to be processed, organized, retrieve or update.
- User: The users are the people who manage the databases and perform different operations on the databases in the database system. They can be end users, data base administrator or application programmer.
- Hardware: Hardware consists of a set of physical electronic devices such as computers (together with associated I/O devices like disk drives), storage devices, I/O channels, that make interface between computers and the real world systems etc.
- Software: It is the set of programs used to handle the database and to control and manage the overall computerized database/
- Procedures: Procedures refer to the instructions and rules that help to design the database and to use the DBMS.

VU Gateway

Module 02: Data Storage Mechanism

A. Data Model Basics

Data model is the basic architectural unit of the databases which determines the structure of the database and determines the way in which data will be stored or organized. It also defines the connection, processing and storage of data in a system. It organize the data and determine how the data elements relate to each other The data model is the map or the design of the database and it is difficult to change when data is inserted.

M. Flat File Basics

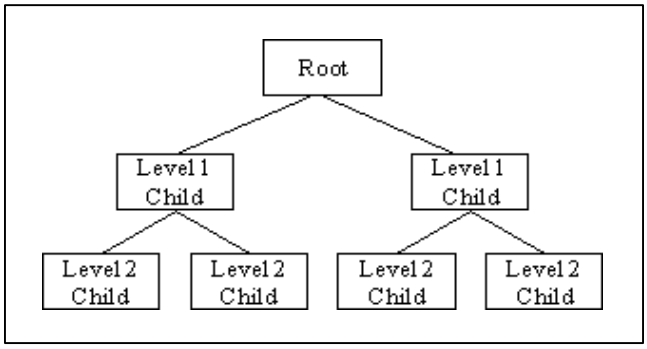
Flat File is a database which is stored on its host computer system as an ordinary file and the complete file has to be read into the memory to process or perform operations. This database contains one record per line and prefers plain text (word or notepad) or ASCII File. The flat file database design put all the data around a single table and data has no interrelationships.

N. How Flat File Works

Data is stored in columns and rows and each row is separated by a comma or tab. Each line of text holds one record. It contains many fields often with duplicate data.There is no relation between data and as more and more records are added to the database, it increases the chances of the data duplication. This redundancy results in reduced integrity of data and it becomes more unreliable. Along with this, the duplication makes it difficult to update the records as there are non-unique records in the database.

ID	Title	First name	Surname	Address	City	Postcode	Telephone
1	Mr	Tom	Smith	42 Mill Street	London	WE13GW	010344044
2	Mrs	Sandra	Jones	10 Low Lane	Hull	HU237HJ	022344033
2	Mr	John	Jones	10 Low Lane	Hull	HU237HJ	022344033

O. Hierarchical Database Basics



Hierarchical database is one of the two main data model proposed by IBM's IMS System. It's a data model in which data is stored in a tree like structure. The data is stored as records (combination of rows and columns) and these records are connected through links. The Hierarchical Data Model is a way of organizing a database with multiple one-to-many relationships. The structure is based on the rule that one parent can have many children but children are allowed only one parent. In order to retrieve data from a hierarchical database the whole tree needs to be traversed starting from the root node.

P. How Hierarchical Database Works

In hierarchical data model, data is retrieved using traversal mean visiting the root node and navigating through hits children. There is one-to-many relationship between root and its child node. Each parent record store the address of its child records and every child records will act as a parent record for its next level.

Q. Network Database Basics

Opposite to the hierarchical model, network model allows each record to have multiple parents and child records. This structure forms a generalized graph structure and this model can be seen as upside down tree. This model is considered to be a very flexible way or representing objects and their relationships with each other. Network database model is easy to navigate because of the linkages between the information and many-to-many relationships.

R. How Network Database Work

There is no concept of parent-child relationship in this type of data model. Each record will store the addresses of all the referencing records. As it support many-to-many relationships between root and child nodes, any record can be linked to any record and thus there is no hierarchy in this type of data model.

VU-Gateway

Module 03: Three-Tier Architecture

A. Introduction to 3-Tier Architecture

Tiers can be defined as the collection of logically related but physically independent components of an application. In this architecture, the application is divided into 3 tiers in which each tier has its own boundary. Each tier contributes towards the whole and several tiers join to make something whole. By segregating an application into various tiers, developers acquire the option of modifying or adding a specific layer, instead of reworking the entire application.

S. Rationale for 3-Tier Architecture

3-Tier architecture came as a solution to the complexity of the application and enterprise scalability, the new architecture purpose three layers and different teams can work on the different layers at the same time thus making the development process easy. Dividing application into 3-Tiers helps avoiding single point of failure in the application and failure of one tier doesn't mean the crash of whole application.

T. External Level (Client Interface)

It is the top most layer of 3-Tier architecture and it is also known as "Presentation Layer". This tier managers the input/output data and their display. In other words it displays what clients view. This tier is responsible for formatting the input/output data into a presentable form. This tier is able to communicate one tier below with the application tier to display information. In simple words, it's a tier which users can access directly such as mobile applications, browsers and forms etc.

U. Conceptual Level (Business Logic)

This tier – also referred to as the application tier – is the middle tier that bridges the gap between the user interface and the underlying database. This tier consists of business and data rules. In this tier the logic of the business is encapsulated (Programming/Class Diagram/ERD). This tier receives the requests for the presentation tier only and interprets or processes the requests as per the defined rules and sends back the results to the presentation tier only. It can receive the request and send the response to the presentation tier only, and it can send request to only the below tier i.e. Internal Level (Data Tier)

V. Internal Level (Data Tier)

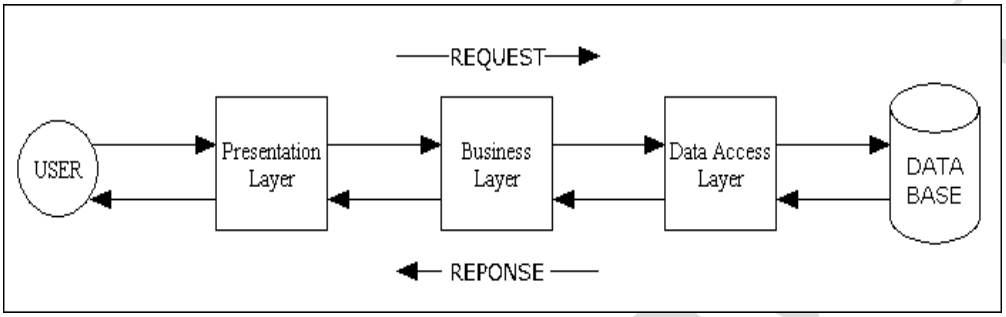
This tier is responsible for storing information and data needed for the system and for optimizing the data access. Data needed by the application logic layer are retrieved from the database, and then the computation results produced by the application logic layer are stored back in the database. This tier can receive requests from Conceptual Level or logic tier only and can send the responses to the said tier only. All the decisions related to the data storage are stored here and these storage mechanisms are independent of other tiers.

VU Gateway

W. Layer Independence

The tiers mentioned above have no interdependence and work exclusively. Every tier is totally unaware of the internal working or the logics of other tiers and the decision made by the tiers are also independent. In simple words:

- Data layer is not aware of the internal working of logic layer
- Logic layer is not concerned about the workings of data layer
- Presentation layer works in total independence for other layers

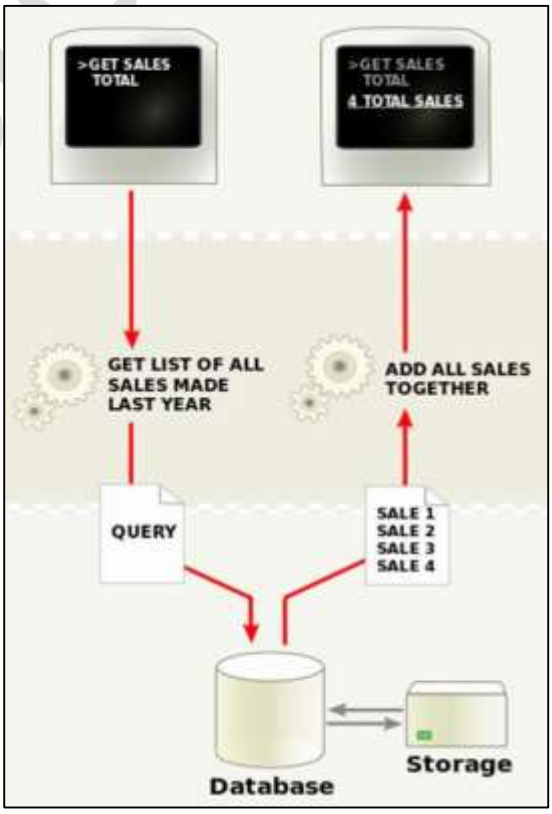


X. Example – Presentation Layer Independence

Consider the following situation:

Query: Get Total Sales

A manager of a company need to run sales query to find out the list of all sales made last year. He will use the presentation tier or the client interface to send this request to the application tier. Suppose this is done by using a form or any other way. This form, browser or mobile app through which the request is being made cannot affect the way the data is retrieved. All other tiers will work in total independence. Same way, all other tiers can't affect the way data is presented in the presentation tier.



Y. Example – Logical Layer Independence

Continuing with the same example:

Query: Get List of all Sales made last year

The structure or the workings of this tier is also exclusive to the logical tier. Using if-then, while or any other structure (i.e. how the query is made) is internal to this tier only and it can't affect the way data is displayed.

© Copyright Virtual University of Pakistan

Z. Example – Data Layer Independence

As shown in the figure 6 above, the application tier will send query to the data tier and it will respond to the application tier only with the requested results. What logic the application has used or it will use is not the concern of the data tier; it will just provide the required results. The application tier will perform the activity as it is programmed to do and will forward the results to the presentation layer which will display the results to the end user. None of the mentioned tiers can affect the working or processes of the other tiers.

AA. Roles in Database Management System (DBMS)

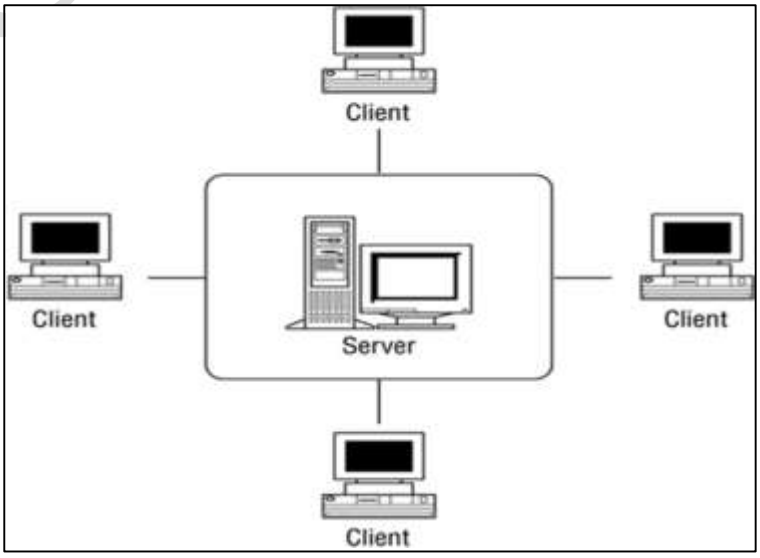
There are following roles in a database environment. The details of them are also given.

- ***Application Programmer*:** application programmer develops the application that provide required the functionality by the end user.
- ***End User:*** The end-users are the 'clients' for the database, which has been designed and implemented, and is being maintained to serve their information needs.

  Database Administrator: Database Administrator (DBA) is responsible for the physical realization of the database, including physical database design and implementation, security and integrity control, maintenance of the operational system, and ensuring satisfactory performance of the applications for users.
- ***System Analyst:*** systems analyst is a person who uses analysis and design techniques to solve business problems using information technology.

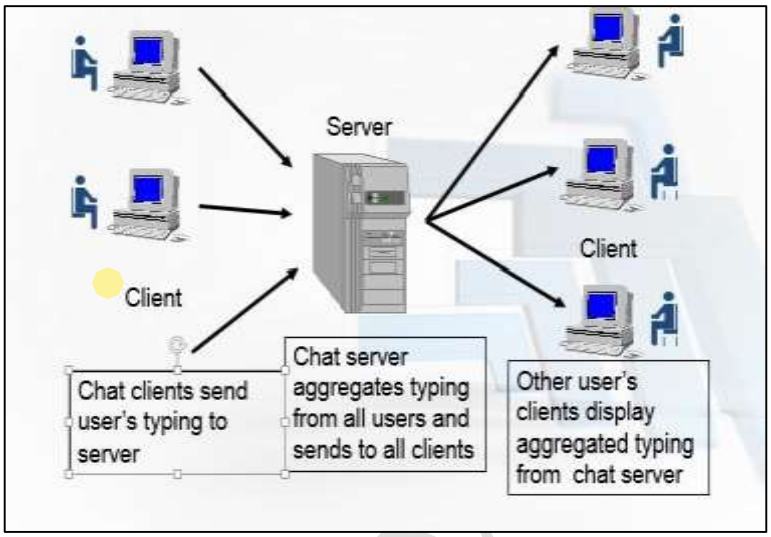
BB. Client Server Architecture

Client-server architecture is architecture of a computer network in which many clients (remote processors) request and receive service from a centralized server (host computer). Client computers provide an interface to allow a computer user to request services of the server and to display the results the server returns. Servers wait for requests to arrive from clients and then respond to them. Server is powerful in terms of processing and the file server, printer server or the email server. The client is connected to the server via communication link. The client interacts with the server when it requires access to any additional functionality that does not exists in its own system client system provides interfaces to access and utilize server resources.



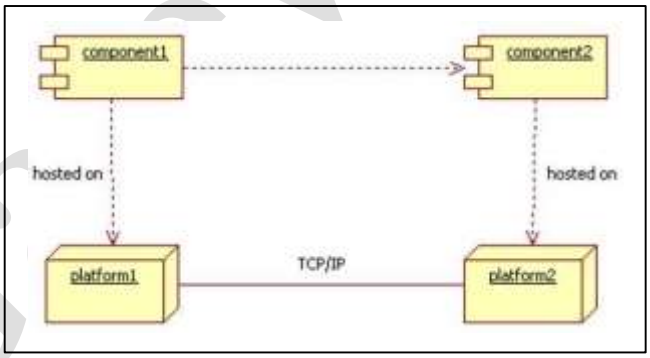
CC. Example – Client Server Architecture

The client server architecture is best elaborate with the example of chatting over the internet where the communication takes places between multiple clients with multiple clients. The figure 8 shown at the right side explains further. The chat clients send the user typed message to the server, the server then aggregate typing from all the clients and forwards it to all the clients, user's clients on the other side display the aggregated typing from the chat server.



DD. Distributed Processing

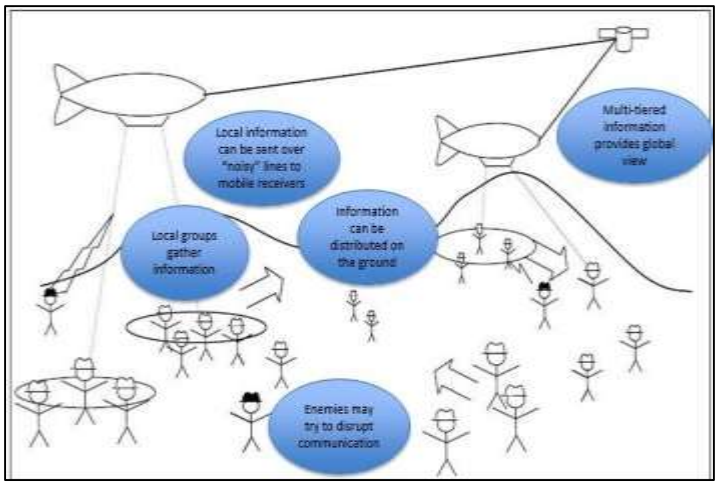
Distributed system is combination of multiple components located on network and these components communicate and coordinate their action by passing messages. These components interact with each other to achieve a common goal. The processing is divided into multiple systems increasing the processing power. In distributed computing,



a problem is divided into many tasks, each of which is solved by one or more computers, which communicate with each other by message passing as mentioned earlier.

EE. Example of Distributed Processing

TO BE DONE BY SHERAZ SB



FF.

Module 04: Fact Finding Technique

A. Examine Document Basics

Examine documents is one of the fact finding techniques and it is the first step in database development. This technique is critical to the design and development of database as it put some light on how the need for a database arose and what symptoms and problems in the current system lead to this decision. It is also helpful in studying the current system and the in the identification of the effected part of the organization. It is also valuable tool for the identification of the data to be collected and reported by the system.

Purpose of documentation	Examples of useful sources
Describes problem and need for database	Internal memos, e-mails, and minutes of meetings Employee/customer complaints, and documents that describe the problem Performance reviews/reports
Describes the part of the enterprise affected by problem	Organizational chart, mission statement, and strategic plan of the enterprise Objectives for the part of the enterprise being studied Task/job descriptions Samples of completed manual forms and reports Samples of completed computerized forms and reports
Describes current system	Various types of flowcharts and diagrams Data dictionary Database system design Program documentation User/training manuals

GG. Documenting the Examine Document

Documenting the existing systems of the documents leads to the identification of the type of data coming in and out of the system. It lays down the foundation of the requirements for the development of databases. Documenting the existing system and documentation identifies the data being handled and the relationship between the data. This relationship explains how the data flow from one part to another thus making a better and efficient system. In the figure on the right there is data about customer, and services requested (problems are categorized into two categories). The purpose of the figure is to show that by examining the documents within organization difference stakeholder holders, information flows are identified within the system which are difficult to understand by listening..

CUSTOMER SERVICE REQUEST

CUSTOMER INFORMATION
Date
Customer Name
Address
Phone

SERVICE REQUEST INFORMATION			
MERCHANDISE PROBLEMS:		DELIVERY PROBLEMS:	
Order not filled		Bad address	
Defective merchandise		Customer not in	
Repair problem		Delayed/lost in transit	
Wrong merchandise sold		Damaged in transit	
Amount charged in error		Mdse. Missing in package	
Credit/refund not issued		Other	
Other			

OTHER INFORMATION
REMARKS:

HH. Conducting Interviews Basics

Interviewing is the most commonly used, and normally most useful, technique for fact finding. It is a technique where the business analyst gathers information by having face-to-face interaction. There can be multiple objectives to using this technique such as fact finding or verifying the

facts, getting more diverse ideas and information, gathering the requirements, involving the end user to generate enthusiasm and to make them feel engaged in the whole project.

II. Types of Interview

There are two types of interview: unstructured and structured. The details of them are as follows:

- *Unstructured Interviews:* These interviews are conducted with a general goal in mind with very few specific questions. The interviewer let the interviewee to drive the discussion and provide direction to the interview.
- *Structured Interviews:* in this type of interviews, the interviewer has specific set of questions. Those questions can be further divided into two categories: Open-ended and Close-ended Question. Open-ended questions let the interviewee respond in any way that seems appropriate with no specific options to choose. Close-ended question restrict the answers to either specific choice or to a very short answers.

JJ. Example of Type of Interviews

Example 01: Unstructured Interview: Consider this interview between a researcher and a caregiver

- Researcher: "What kinds of illnesses do children in this village get?"
- Caregiver: "Well, diarrhea is the most common; also cough and cold, measles, fever, typhoid."
- Researcher: Are there any other during cold and hot weather?
- Caregiver: Pneumonia and whooping cough are very common in winter
- Researcher: Can you tell me something about measles?
- Caregiver: During measles it is very good to give cold water because it is very hot. The child should be kept away from other children
- Researcher: Why the child should be kept away from other children's?
- Caregiver: because measles can spread to other children very easily. Once a child get measles all the family get it
- Researcher: How does measles spread from child to child?
- Caregiver: I think it must happen through smell and sweat. May a child touches a thing which a sick child touches, I am not sure

Example 02: Structured Interviews: Consider the following questions:

- Could you tell me the tasks you do to prepare salaries per month?
- How do you know person monthly salary?

KK. Observing the Enterprise in Operations

Observation is one of the most effective fact finding technique as it give analyst a chance to participate in or watch the person perform the activity which results in deep understanding of the system. This technique is particularly useful when the validity of data collected through others methods is a serious concern or the system is too complex that the person or the end use in not able to explain it properly. To make sure the success of observation, it is necessary to know as

© Copyright Virtual University of Pakistan

much about the individual and activity to perform as possible. Here the question like, when low, normal and peak periods for the activity being observed.

LL. Have and Have Not's for Observing

The observation should be recorded immediately or simply in parallel with the observation. The observer must not wait the observation to end and then write the complete process; rather the observer must take notes in the form or bullets during the observation in order to avoid losing any point. The observer must not interrupt or ask too many question from the end user while he is performing the activity otherwise the activity will get tempered and the observer will get the real information.

MM. Research

It is very much valued to research the solution for the problem in hand. Internet, journals, reference books are good sources of information about how other have solves related problems or what solutions are available. It can save a lot of time as we can get ready to deploy solutions to fulfill the requirements. It also a helpful tool in validating the information gathered thorough observation as the chances are there that the actual execution of the activity is different from what is observed.

NN. Questionnaire

Another useful method for fact finding is to conduct surveys through questionnaires which are specific purpose document to gather information from a large number of people while maintaining some control over the responses. There are two types of questions that can be asked in a questionnaire and the details are given below:

- *Free- Format Questions*: it offers greater freedom to the respondent in answering question as the respondents have to answer the question in the space provided after the question. The responses are more subjective in nature and difficult to tabulate.
- *Fixed-Format Questions*: These kinds of questions require specific responses and the individuals have to choose from the options available. The responses are more specific which make the responses much easier to tabulate. The disadvantage of this kind of questions is that the researcher doesn't get additional information that might be worth knowing.

OO. Example of Questionnaire

Consider the following requirement and observe the question in the questionnaire shown in the figure 12 below:

Requirement: What are current trends among students in terms of interest in media programs?

VU-Gateway

1. Are You: a. Male b. Female	2. When was the last time you saw a file, what was it?
3. How old are you? Years……. Months……	4. How many hours a day you spend watching, reading or listening to the following a. TV b. Radio c. Internet d. Print (Magazines / Newspaper)
5. Who do you live with at home? (be Specific)	6. Please list your top three TV programs a. b. c.
7. Do you have a part-time job? a. Yes b. No c. Paid d. Voluntary If so, please describe what you do	8. List in order of preference (1 being more preferred, 5 being least) which genres (types of programs) you watch: a. Sports b. Soap c. Sitcom d. Documentaries e. Film
9. Please list you top 3 favorite food? a. b. c.	10. You are going to help organize some kind of music even for your year group: what type of music / bands would you want to play?

Figure 1: Sample Questionnaire for Fact-Finding

VU-C

Module 05: Implementing Fact Finding Techniques

A. From Fact-finding to Requirement Writing

Requirement writing is the very next step after fact finding in the cycle of database design and development. The requirements defining the systems are to be converted in written format. Along with the system definition it also contains and explains the data flow in the system and the relationships between the data. The requirement or the specification document describes in details the data to be held in database and how the data is to be used (i.e. the processes and the relationships). These requirements are specific and measureable in nature. The system requirement like how fact the transaction must run or how many transactions must be processed in specific interval of time form the basis of number of decision in database design and development.

PP. Example of Translating Interview to Requirements

Consider the following example in which the interview asks questions to get the requirements for high level database for rental system of a certain plaza. Following the interview are the requirements gathered through the interview.

Interview:

1. Interviewer: How many building types are there for rent?
   Interviewee: Primarily Apartments, some Offices and few Shops
2. Interviewer: How many are total apartments, total offices and total shops
   Interviewee: Ahhhh… yes 15 Apartment, 5 offices and 3 shops
3. Interviewer: How many floors are there in the plaza?
   Interviewee: 6 floors but we have apartments only on 3 floors and on remaining floors we have shops and offices
4. Interviewer: Do we have fix rent for each apartments or does it varies from Floor to Floor
   Interviewee: Rent varies from Floor to Floor of apartment.
5. Interviewer: Did you send rent bill / receipt at the end of month?
   Interviewee: We usually send bill for two-months but receive installments on monthly basis also
6. Interviewer:, can you tell me specific data you store about customer into the system
   Interviewee: We usually ask for name, address and phone no
7. Interviewer: Did you assign any number to customer
   Interviewee: Yes, I miss out
8. Interviewer: If multiple apartments / shops / offices are rented to same customer, then do we have same contract no and customer no
   Interviewee: Good question, let me think….pause, we usually assign different contract no but one bill to such case.

Requirements:

VU Gateway

In a plaza renting system, there are three types of buildings primarily apartment, offices and shops. There are multiple floors in a plaza and for 3 floors apartments are allocated but for remaining there can be mix of shops and offices. When customer is registered usually customer id, name, phone and address is stored as profile and bill is generated for the rent on bi-monthly basis but customer has an opportunity to pay the bill on monthly basis. Contract is signed and uploaded against every building type. Contract id assigned to every contract. It is possible that a customer may rent multiple building types, in this case different contract no is assigned to same customer. One Bill is generated for customers with multiple building types are generated.

QQ. Example of Translating Questionnaire in Requirements

Following is the sample questionnaire to elaborate the translation of data gathered through questionnaires into requirements.

Questionnaire:

1. What is price (person person) range of food offered?
    - 500-1000
    - 700-1200 **(checked)**
    - 1000-1500
2. Is cost of dish calculated on the basis of?
    - Cost of ingredient only**(checked)**
    - Cost of ingredient and electricity also
    - Cost of ingredient plus % of profit
3. Is % of Profit?
    - 20%**(checked)**
    - 15%
    - 10%
    - None of the Above
4. Are ingredients and Quantity mentioned in?
    - Menu
    - Recipe **(checked)**
    - Restaurant Manual

Requirements: the requirements can be concluded as the following:

In a restaurant management system there are many dishes offered either as individual serving or buffet. Price range between 700-1200 is targeted which will include drinks and taxes also. Cost of every dish is calculated individually depending on ingredients involved only. Each ingredient is listed in recipe and quantity is also mention. Cost is calculated as per market rate of ingredient with 20% profit to cater for management and miscellaneous cost.

RR. Defining System Definition from Requirements

System definition is defined by listing down the details of the whole system. These details when properly concluded form the basis of decision regarding the scope and the boundaries, the

VU-Gateway

architecture and the design of the system to be developed. Concept definition artifact lays down the foundation of system definition; concept definitions are activities in which the needs and the requirements of all the stakeholders are closely examined before defining the system. The concept definition is the articulation of System-of-Interest (Sol) – the model of system on the basis of needs and requirement – which is a collective set of all elements of any system being considered by the lifecycle.

SS.Process of Extracting System Definition

System definition is based on the concept definition, a deep understanding of concepts, needs, expectations and stakeholders' requirements. System is further divided into subsystems through top-down decomposition of system-of-interest in order to reach the exact destination. The same model is shown in the Figure 13 on the right side. The needs and the concepts along with the stakeholder requirements are gathered to form a system-of-interest and this is further divided into subsystem to fulfill the desired requirements.

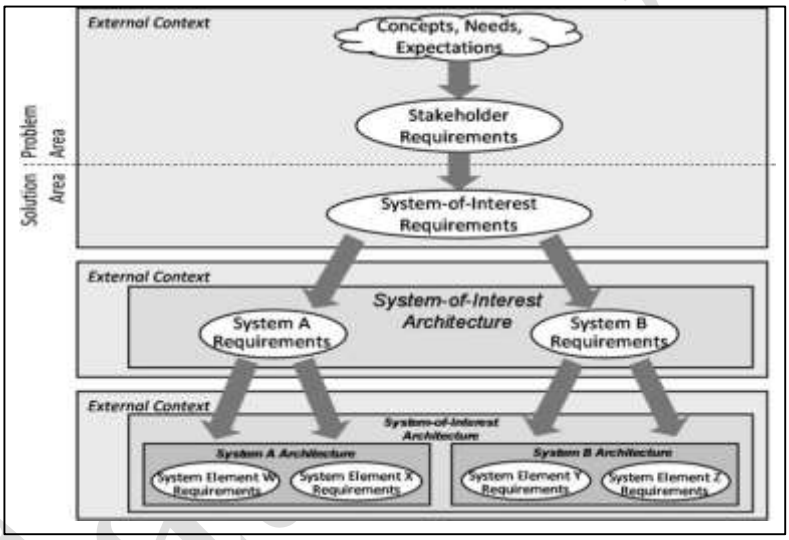


Figure 2: Process of Extracting System Definition

The system should list the entire important feature for a database system and these should include the followings:

- Initial database size
- Database rate of growth
- The types and average number of record searches;
- Performance
- Security;
- Backup and recovery
- Legal issues etc.

VU-Gate

Module 06: Process to Database Design

A. Motivation for Requirements

The requirement gathering process identify the actual need, purpose or the potential update need of a system and these requirement are then base the foundation of system design and development or system update. The requirement gathering process is a non-technical process and it is just the communication of the requirement. This communication is important because this is the thing on which the development of the required system or software is dependent.

The requirement which can be defined as the conditions or the capability, to which a system must conform, can be related to the functionality, usability, reliability, performance or the supportability of the system. There are few challenges in gathering or documenting requirements, discussed in the following lines.



Figure 3: Motivation for Requirement Gathering

TT. Major Challenges in Requirement

The major challenges can in requirement documentation or gathering are explained below:

Missing Requirement: the person gathering or documenting the requirements can miss some important requirements because of human error or any other issues like poor observation, poor analysis of the requirements, the explaining the facts during fact-finding phase (the interviewee, the performer etc.) is unable to explain the fact properly and likes.

Wrong Requirements: there are chances that the requirements are wrongly communicated, exhibited or misinterpreted during requirement gathering and documenting phase.

Changing Requirements: the requirement can change over the period of time. The change can occur in data that is to be processed by the system, the flow of data, and the relationships, the

features required by the end users or the delivery schedule disturbing the development process. The attributes of the requirements are explained in the following parts.

UU. Attributes of Software Requirements: - Verifiable

The requirement should be communicated in such so the testing team should be able to verify it. The requirement will be considered as verifiable if there is some process through which a person or a machine can check that the software product meets the requirement. The following example explains the concept:

- Requirement: Loading time should be as minimum as possible?
- Question: What Qualifies to be minimum Loading Time?
- Implication: Not Verifiable!!
- Solution: Loading time should be 2 sec

X. Attributes of Software Requirements: - Completeness

A complete requirements specification must precisely define all the real world situations that will be encountered and the system's responses to them. It must not include situations that are subjective in nature, non-verifiable or non-specific. In simple words, there is no room for guess work in documenting the requirements. Consider the following example:

- Requirement: On Power Loss, backup power should support Normal Operations
- Question: What is definition of Normal Operation?
- Implication: Subjective, Incomplete
- Solution: On Power Loss, backup power should support Normal Operations for 30 min

WW. Attributes of Software Requirements: - Consistent

The requirement is consistent if it is not in conflict with any other existing requirements. Conflict may arise due to variation in language patterns, use of different vocabulary or multiple descriptions of a single requirement. The requirement must not oppose other in terms of physical or functional requirements. The vagueness in the requirement is the quality destroyer. For example: the requirement says perform function X after both A and B has occurred, or perform function X after A or B has occurred. The requirement stated in such way may leads to failure. Another example:

- Requirement 01: The customer support should be ITIL compliant
- Requirement 02: The network support should be ISO – ITIL 3.0 compliant
- Question: Is ITIL and ITIL-3.0 same or different?
- Implications: No Consistency
- Solution 01: The customer support should be ITIL 3.0 compliant
- Solution 02:The network support should be ISO – ITIL 3.0 compliant

XX. Attributes of Software Requirements: - Traceability

Consider the following example:

- Requirement: System must generate a batch report when batch is completed and a discrepancy report when batch is aborted

VU Gateway

- Issue 01: How uniquely would you be able to identify the unique batch report?
- Issue 02: What if requirement has changed and discrepancy report is not required?
- Implication: Zero Traceability
- Solution 01: System must generate a batch report when batch is completed and aborted – Assign Unique Requirement Number
- Solution 02: System must generate a discrepancy report when batch is aborted or completed – Assign Unique Requirement Number

Now understand the concept. A requirement is traceable if both the origins and the references of the requirements are available. Traceability of the origin or a requirement can help understand what modifications have been made to the requirement to bring the requirement to its current state. Traceability of references is used to aid the modification of future documents by stating where a requirement has been referenced. By having foreword traceability, consistency can be more easily controlled. Making requirements traceable is also useful in coding and testing of the system.

YY. Requirement Engineering Discipline

Requirements engineering emphasizes the use of systematic and repeatable techniques that ensure the completeness, consistency, and relevance of the system requirements [Sommerville 1997a]. Requirement engineering is a set of activities concerned with identifying and communicating the purpose of a software-intensive system, and the contexts in which it will be used. Hence, Requirement Engineering acts as the bridge between the real-world needs of users or customers and the capabilities and opportunities afforded by software or system. Specifically, requirements engineering is divided into requirement management and requirement development which is further divided into requirements elicitation, analysis, specification and verification.

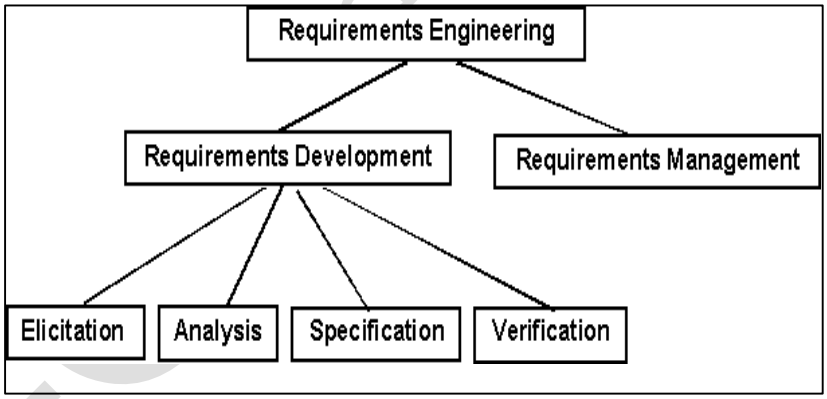


Figure 4: Sub Disciplines of Requirement Gathering

ZZ. Requirement Management

Requirements management is the process of scheduling, coordinating, and documenting the requirements engineering activities (that is, elicitation, analysis, specification, and verification) [Dorfman 1997a]. This discipline takes into account different strategies to keep the project scope within the baseline. Besides, it also monitors and controls the scope of the project and provides a mechanism to handle the proposed changes in the project scope. This discipline of the requirement engineering reviews and analyzes the changes to know the impact of them on the product line (impact on delivery schedule etc.) while communicating it to the relevant stakeholders.

AAA. Example of Requirement Management

Consider the following example of requirement change from the client side:

- Baseline Requirement: A - 22-May-2015
- Development as per A: 2-June-2015
- Requirement Change by Client – B: 10-June-2015
- Demo Day: 15-June-2015
- Client: This is not what I was expecting
- Reason: Client referring to B, Development to B!!

BBB. Requirement Development (RD)

Requirement development is a part of requirement engineering in which the requirements are gathered and then turned into specification of what system must do, or the scope or features of the software system. Requirement development activities are further divided in four activities namely Elicitation, Analysis, Specification and Verification (all of them are explained later). While defining the scope of the project, RD specifically identity and communicate what not to deliver. The artifact of RD process is the baseline requirements (defined schedule).

CCC. Requirement Elicitation or Gathering

Requirement elicitation is all about gathering the system's requirement and this have been discussed in detail in lesson 04 fact finding techniques. The aim is to apply all the techniques here to get the real requirements of the client. One technique is not just enough for this purpose because every requirement gathering has its own pros and cons and to eliminate the limitation of a single technique, more than one technique must be used. It could be interview, questionnaire, observation or any mixture of them as per the need.

DDD. Project Business Requirement

There must be a baseline for a scope in order to manage the scope in a proper manner. The priority of the scope is to be defined along with the baseline to deliver. This prioritizing helps the development team in scheduling their activities to deliver the project scope in time. This is all about defining and communicating the product roadmap. Let's take an example of this concept under next heading.

EEE. Example of Project Business Requirement

Product Roadmap:

- Total Scope of Project: 20 Features
- High Priority Features: 8 (Agreed by Client and PM)
- Medium Priority Features: 7
- Normal Priority: 5
- Release – 1: 8 Features with X Date
- Release – 2: 8 + 7 Features with Y date (medium priority)
- Release – 3: 8 + 7+ 5 Features with Z Date (normal priority)

VU Gateway

FFF. Get User Involvement

Lack of user input can make a project very challenging. User involvement is of utmost important at every stage of system development. First of all, at the time of requirement gathering it is important to get the users or all stakeholders involve in order understanding the real issues in the current system and the potential solution to be developed. By understanding the skills and capabilities of the users, it would be possible to tailor the user interface to the need of particular type of users to ensure optimum usability. The concept of 'Product Champion' is worth mentioning here. The product champion is the person who is officially responsible for the delivery of the system. This person helps the stakeholders and the project manager reach a shared vision for a product, and then defines and initiates the product within that vision. The product champion have technical competence, knowledge about the company and markets and he is able to get as much knowledge as possible that if the client's representative leave, valuable knowledge doesn't leave with him.

GGG. Define Quality Metrics

A metric is a measure and combining it with the quality means the measures of quality. The quality is obviously what is defined by the customers. If the system provides what customer needs, that can be called a quality system. But there is much more than that. The focus of the quality metrics are non-functional requirements that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors. Broadly, functional requirements define what a system is supposed to do and non-functional requirements define how a system is supposed to be. The few quality metrics for a system are:

- Reliability
- Robustness
- User Friendliness

HHH. Example of Quality Metric – Reliability

Reliability can be defined as the capacity of the designed or developed system to perform as it is required to perform over time. It can also be defined as resistance to failure over time. Reliability is all about stability of the developed system or its ability to perform what it is designed to perform. Following example will explain further:

- Example: We have developed a program for a Web server with the failure intensity objective of 1failure/100,000 transactions. During testing, the program runs for 50 hours, handling 10,000 transactions per hour on average with no failures occurring.

III. Example of Quality Metric – Robustness

Robustness is the ability of a computer system to cope with errors during execution. Robustness can also be defined as the ability of an algorithm to continue operating despite abnormalities in input, calculations, etc. The question or the check for this metric is the question that does the system crashes on abnormal inputs. For example: How algorithm behaves in case of abnormal input. If user input character A instead of numeric value then does system crash?

VU-Gateway

JJJ. Example of Quality Metric – User Friendliness

User Friendliness is synonymous with the usability and this term can be defined as the ease of use or the learn ability of the developed system. User friendliness is the extent to which the developed system can be used by the end user to get the desired results with effectiveness and ease of use. Most clients reject the software on the basis of the interface that is not too user friendly. This is where the requirement gathering becomes most important. The solution to this issue is to get the details on how the client wants it to be. For example the color scheme of the software, first ask the client for the color theme, get the color code or the sample image and then use the mockups of that image.

KKK. Requirement Analysis

After gathering the requirements the next step is to analyze the requirements. The basis of this analysis is the attributes of the system requirements mentioned in the same lesson above (heading C – F). This stage breaks down functional and non-functional requirements to a basic design view to provide a clear system development process framework. A thorough requirement analysis process involves various entities, including business, stakeholders and technology requirements. This stage also identifies the conflict with the base line as the prototype is made and the further development needs are clear at this stage.

LLL. Requirement Prioritization

At this stage the priority of the requirement is defined. The priority is defined in coordination with the development team and the client and the priority is based on some statistics of facts. The limitations of the project are also identified here. This prioritization helps in the stage-release of the system as the development of the system is divided into stages based on the priority. The same example mentioned in the section M of the same lesson also fits here where the requirement are prioritized and the release planning is done on the basis of it. The approach is to categories requirements into High, medium and low on some statistical basis.

MMM. Requirement Specification

Requirement specification is the result of requirement analysis. Requirement specification established the basis for the agreement between clients and the development team on what the software product is to do and what not to do. The requirements are further classified into functional and

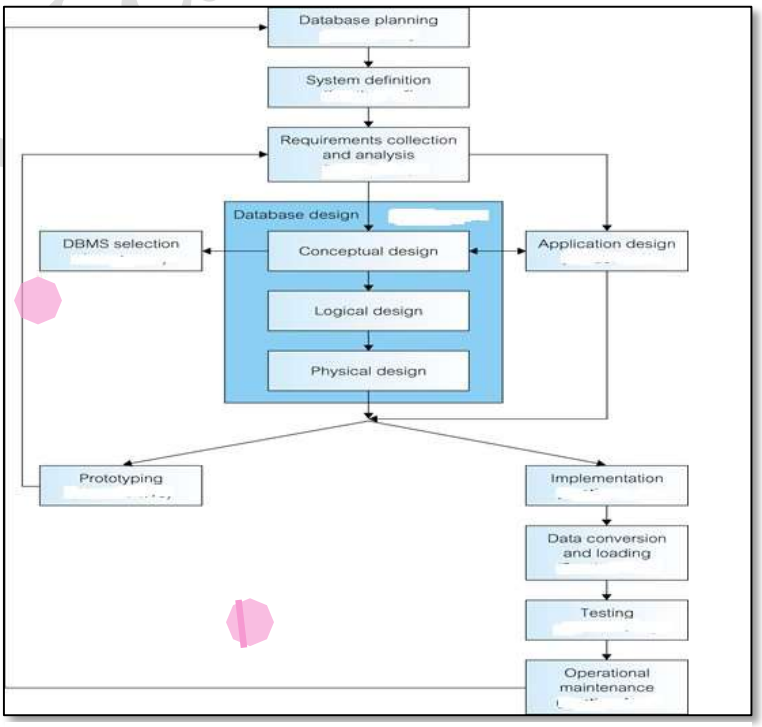


Figure 5: Process to Database Design

nonfunctional categories and recorded in mutually agreed document. This document is called Software Requirement Specification or SRS. This document enlists enough and necessary requirements that are required for the project development. Clear and thorough understanding of the system to be developed is making sure and this is achieved through continuous communication between client and project development team.

NNN. Requirements Verification

The correctness and the completeness of the system requirements is again verified at this stage, just to make sure that the system in the manufacturing process will ensure user need and requirements. Verification can also be seen as the process of checking that the software meets the specification. The goal of requirement verification is to ensure that requirements are good enough to proceed to design phase and development phase. This stage consists of thorough inspection and verification of system requirement as the defect and bug fixing later cost too much. Defect is defined as the wrong definition of the client's requirements. Defect is not something that is not working, it is simply working but not required part of the system and this can be detected in production phase. On the other side, bugs can be defined as the noncompliance with the agreed behavior, or software is not doing what it is supposed to do. This is a deviation from the requirements of the system.

VU-Gateway

Relational database can be defined as a database whose organization is based on the relational model of data in which all the data is presented in the forms of tuple (also known as a record or a row) which are further grouped into relations. The relational model was first proposed by E. F. Codd in his seminal paper 'A relational model of data for large shared data banks' (Codd, 1970). There are some rules proposed by E. F. Codd which any database must follow to be called a relational database.

A. Codd Rule No. 01: The Information Rule

"All information in the relational database is represented in exactly one and only one way—by values in tables."

All the information (data or Metadata) must be stored in a table format. This is the only way data is to be stored. More specifically the storage mechanism should follow the row and column format. This rule is an informal definition of a relational database and indicates that every piece of data that we permanently store in a database is located in a table.

B. Codd Rule No. 02: Guaranteed Access Rule

"Each and every datum (atomic value) is guaranteed to be logically accessible by resorting to a combination of table name, primary key value, and column name."

This rule emphasizes the importance of the primary keys in locating the data in any database. The table name locates the correct table, the column name finds the correct column, and the primary key value finds the row containing an individual data item of interest. In other words, each piece of data is accessible by the combination of table name, primary key value, and column name.

C. Codd Rule No. 03: Systematic Handling of Null

"NULL values (distinct from empty character string or a string of blank characters and distinct from zero or any other number) are supported in the fully relational RDBMS for representing missing information in a systematic way, independent of data type."

Null cannot be considered as zero or empty, it is just something we just don't know and lack of information does not mean no information. The NULL values in a database must be given a systematic and uniform treatment. This is a very important rule because a NULL can be interpreted as one the following − data is missing, data is not known, or data is not applicable.

Example:

- Question: How many Books Ali have?
- Answer: May be 5 or not known (null)
- In Database column representation not known will be using NULL not using any numeric value
- Conclusion: There will be representation of null beside numeric value in database

© Copyright Virtual University of Pakistan

D. Codd Rule No. 04: Active Online Catalogue

"Data Dictionary and database description must be available in database"

The structure description of the entire database must be stored in an online catalog, known as data dictionary, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself. The system must support an online, inline, relational catalog that is accessible to authorized users by means of their regular query language. That is, users must be able to access the database's structure (catalog) using the same query language that they use to access the database's data.

E. Codd Rule No. 05: Powerful Language

"A database can only be accessed using a language having linear syntax that supports data definition, data manipulation, and transaction management operations."

A linear syntax can be defined as a language which is parsed from left to right which implies the ability to write code without the use of line-feed or carriage-return characters. Line-feed and carriage-return are much closed concepts and can be lumped together and these are special character or sequence of characters signifying the end of a line of text and the start of a new line.

OOO. Example of Power Language

- Linear Syntax:

  Void main () { cout<< "Hello";}- **C++ Language**
- Language rely on semi column and parenthesis to separate code block
- Line Feed or Carriage Return (i.e. moving to next line) is not required to separate code block

F. Codd Rule No. 06: View Updating Rule

"All views that are theoretically updateable are also updateable by the system."

This rule deals with views, which are virtual tables used to give various users of a database different views of its structure. It's one of the most challenging rules to implement in practice, and no commercial product fully satisfies it today. User can be given access to selected columns as per rights and privileges. Views are created on the basis of tables. Views should be updateable without any restriction and should be updateable like tables. SQL have limitation to update views and view with single table and without group by clause are updateable only.

G. Codd Rule No. 07: Relational Level Operations

"Insert, update, and delete operations should be supported for any retrievable set rather than just for a single row in a single table."

© Copyright Virtual University of Pakistan

The system must support set-at-a-time insert, update, and delete operators. This must not be limited to a single row, that is, it must also support union, intersection and minus operations to yield sets of data records.

H. Codd Rule No. 08: Physical Data Independence

"The way the data is stored physically must be independent of the logical manner in which it's accessed."

This rule says that Changes to the physical level (how the data is stored, whether in arrays or linked lists etc.) must not require a change to an application based on the structure. This is saying that users shouldn't be concerned about how the data is stored or how it's accessed. In fact, users of the data need only be able to get the basic data they need.

I. Codd Rule No. 09: Logical Data Independence

"Changes to the logical level (tables, columns, rows, and so on) must not require a change to an application based on the structure."

The logical data in a database must be independent of its user's view. How a user views data should not change when the logical structure (tables structure) of the database changes. This rule is particularly difficult to satisfy. The crux of this rule is that the business logic and storage mechanism works in total independence with the way user view the data and the user should be unaffected with any change in storage mechanism or business logic.

Example:

- Structure of Table – Employee (id, name, DOB)
- Application Program Access: name, id, DOB from Database
- Database Structure need no change

J. Codd Rule No. 10: Integrity Independence

"A database must be independent of the application that uses it."

All its integrity constraints can be independently modified without the need of any change in the application. This rule makes a database independent of the front-end application and its interface. Changes to the logical level (tables, columns, rows, and so on) must not require a change to an application based on the structure. Logical data independence is more difficult to achieve than physical data independence. Integrity rules should be stored in the data dictionary. Primary key constraints, foreign key constraints, check constraints, triggers, and so forth should all be stored in the data dictionary.

K. Codd Rule No. 11: Distribution Independence

"The database language must enable application programs and terminal activities (storage or data retrieval) to remain logically unimpaired whether and whenever data are physically centralized or distributed."

This rule says that the database language must be able to manipulate data located on other computer systems. The end-user must not be able to see that the data is distributed over various

VU Gateway

locations. Users should always get the impression that the data is located at one site only.User have no concerns with the internal storage strategy. This rule has been regarded as the foundation of distributed database systems.

L. Codd Rule No. 12: Non-Subversion Rule

"If a relational system has or supports a low-level (single-record-at-a-time) language, that low-level language cannot be used to subvert or bypass the integrity rules or constraints expressed in the higher-level (multiple-records-at-a-time) relational language."

In simple words, there should be no backdoor to disable the integrity and constraints defined. This rule requires that alternate methods of accessing the data are not able to bypass integrity constraints, which means that users can't violate the rules of the database in any way. If low-level access is allowed, it must not bypass security nor integrity rules. For example, a backup or load utility should not be able to bypass authentication, constraints, and locks etc. in Oracle there is violation of this rule.

PPP. Database Schema

Database schema is better explained as the group of database objects like tables, indexes, triggers and constraints that are related to each other. It can also be referred as a blueprint of how the database is constructed or the structure of the database. Schema is a logical way of grouping database objects like tables, indexes and views. Usually the schema belongs to a single user or application but a single database can hold multiple schemas belonging to different users or applications. Schema groups the tables by owner thus implementing first level of security as the user can see only the tables that belong to him. Schemas have privileges to have authorized access to Objects and users can be assigned a username and password to have authorized access. A database schema defines its entities and the relationship among them. It contains a descriptive detail of the database, which can be depicted by means of schema diagrams. It's the database designers who design the schema to help programmers understand the database and make it useful.

QQQ. Database Schema Example

Consider the following example in which an admin creates a schema, create a user and sets a password. This user named test in the example can create database objects and can assign rights to other database users.

- Admin Create Schema: Test (username)
- Admin assign Password: 123
- Admin grant privileges to Test
- Test: Login with password
- Test: create table, create constraints, create functions, create triggers – Database Objects
- Test can grant Privileges to other users Database Users

RRR. Introduction to Keys

Each table in a database contains multiple numbers of rows and each row is referred to as one database record. Each record or the row is assigned a unique number to manage the record

© Copyright Virtual University of Pakistan

reference which is known as a key. This key equip or database record with a unique identification which can be easily tracked.

SSS. Structured Query Language

SQL, Structured Query Languages a special-purp programming language designed for managing data held in a relational database management system (RDBMS) and it is considered to be a declarative language. This language is composed of a set of commands that enables creating database and table structures, data manipulation and administration. There are subsets of SQL which are as follows:

- Data Definition Language (DML)
- Data Manipulation Language (DML)
- Data Control Language (DCL)
- Data Retrieval Language (DRL)

TTT. Data Definition Language (DML)

Data Definition Language (DDL) is a standard for commands that define the different structures in a database and is responsible for creating database objects. DDL statements create, modify, and remove database objects such as tables, indexes, and users and all the objects are the part of data dictionary. Common DDL statements are CREATE, ALTER, and DROP.

- CREATE: to make a new database, table, index, user or view.
- ALTER: To modify an existing database object.
- DROP: To destroy an existing database, table, index, or view.

UUU. Data Manipulation Language (DML)

Data Definition Language (DDL) is syntax similar to a computer programming language that is responsible for manipulating Data in Database Table. More specifically, this language is used for selecting, inserting, deleting and updating data in a database. DML statements are used to work with the data in tables and are applied at the table level. DML comprises of the statement that can modify the stored data in tables but not the database schema or objects in data dictionary. Some common DML commands are:

- INSERT: to add new rows to a table.
- UPDATE: to change values already in a table.
- DELETE: to delete the rows in a table

VVV. Data Control Language (DCL)

Data Control Language (DCL) is syntax similar to a computer programming language that is responsible for controlling the access to the data stored in a database. This is used to create privileges to allow users access to, and manipulation of, the database. Two important commands of DCL are GRANT (to give access rights) and REVOKE (to withdraw the access rights). It is also used to control transactional processing in a database and for this purpose the common commands are:

- COMMIT: to apply the transaction by saving the database changes.

© Copyright Virtual University of Pakistan

- ROLLBACK: to undo all changes of a transaction.
- SAVEPOINT: to divide the transaction into smaller sections. It defines breakpoints for a transaction to allow partial rollbacks.

WWW. Data Retrieval Language (DRL)

Data Retrieval Language (DRL) is a command to retrieve data from a database object in the desired format. This is the most popular / flexible and the only way to retrieve data from a database. Command used for this purpose is called SELECT which allow us to specify the type of information which we want to retrieve.

VU-Gateway

Module 08: Conceptual Data Modeling and Entity Relationship Diagram Overview

A. Concept of Entity Relationship Diagram (ERD)

Entity–relationship modeling was developed by Peter Chen and published in 1976; it serves as building block of relational database design. Entity relationship diagram is a graphical representation of the relationships between data in a database. It is the result of using systematic process and it just only visualize the business data instead of defining the business process. In very simple terms, ERD is a visual representation of data that describes how the data is related to each other.

XXX. Rationale for ERD

It is the first diagram in database design which gives a higher level description of the system and actually visualizes the system requirements. It is probably the easiest way to describe the interaction or the relation between the different components by using different notations and to get a picture of functionality needed for the system. The data is represented as components (entities) that are linked with each other by relationships that express the dependencies and requirements between them thus making them easy to understand.

YYY. Components of ERD

There are three main components of ERD and these are:

- Entity
- Attributes
- Relationships

ZZZ. Entity & Attributes

The word entity is rooted from the Latin word "en" which means being. Entity is name of place, person or thing about which something can be stored in a system. An entity can be a real-world object that can be easily identifiable. For example, in a school database, students, teachers, classes, and courses offered can be considered as entities. All these entities have some attributes or properties that give them their identity. An entity set is a collection of similar types of entities.

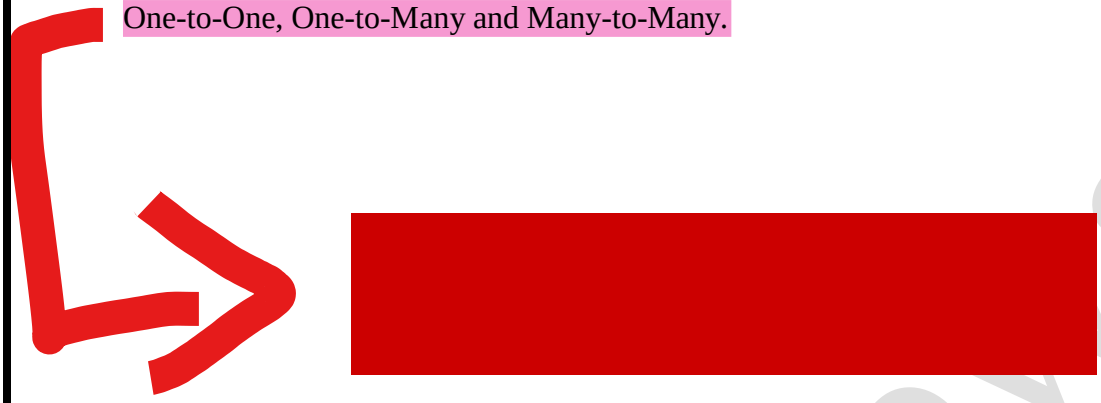
Entities are represented by means of their properties, called Attribute or Column. All attributes have values which are the qualities or data about Entities that is to be stored. An Attribute describes a property or characteristics of an entity. Continuing with the above example, a student entity may have name, class, and age as attributes. Attribute is the smallest storage unit of any database.

AAAA. Relationships

Relationship represents how data is connected among entities in a given System. The association among the entities can also be termed as relationships. In our school example, the two entities e.g. student and course have an association or relation with each other as student enroll in a course. Interaction among entities is captured using relationships.

BBBB. Process to Create ERD

The first step in creating the ERD is to define or determine the entities from the data. The second step is to chalk out the relationships and interaction between the entities identified in the very first step. The third step is to identify the cardinalities which is also known as degree of relationship which is the number of entities in an entity-set which are associated (or linked) to the number of entities in another entity-set. There are three degrees of relationship, known as: One-to-One, One-to-Many and Many-to-Many.



VU-Gateway

Module 09: Entities, Attributes and Relationship

A. Extracting Entities from Requirements

In the database design process, designers first produce requirements specifications from users, then transform the requirements into a formal representation schema. The Entity Relationship Diagram (ERD) model is one of the most popular of these formal representation schemata. It is a graphical way of displaying entity, relationship, and attributes types, which incorporates some of the important information about the situation, and it is considered to be a very natural and easy-to-understand way of conceptualizing the structure of a database.

Consider the following scenario and observe how the entities are being determined from the requirements.

Scenario:

In a Building apartment renting scenario, there are apartments, buildings and customers. There are multiple floors in the building and on each floor there are multiple apartments, floor can have zero or no apartment. Each apartment can be rented at most one customer but customer can rent out multiple apartments from same building and apartment can be available on multiple floors or same. At the end of month a receipt is generated against which a rent is deposited.

The following entities can be derived from the above stated scenario:

- BUILDING
- APARTMENT
- CUSTOMER
- FLOOR
- RECEIPT

CCCC. Extracting Attributes from Scenario

Considering the same scenario, attributes against each derived entities can also be determined. Recall the concept of attributes; an attribute describes the facts, details or the characteristics of an entity. From the above scenario, the following and many other attributes can be defined.

Entities & Attributes:

Entity	Attributes
Building	Building Name (bname), Address, Phone Number
Apartment	Apartment ID (aid), Covered Area, Status (available/rented), Rent…
Customer	Customer ID (cid), Name, Contact, CNIC
Floor	Floor Number, Number of Apartments
Receipt	Receipt ID (rid), Apartment Number, Date of Receipt, Amount

The ability to find meaningful names comes with fundamental understanding of what the model represents. Since the requirement specification is a description of a business, it is best to choose

VU-Gateway

meaningful business names wherever that is possible. If there is no business name for an entity, you must give the entity a name that fits its purpose in the model.

DDDD. Primary Key

The primary key of a relational table uniquely identifies each record or row in the table. As its name suggests, it is the primary key of reference for the table and is used throughout the database to help establish relationships with other tables. It usually comprises of a single table column, but may consist of a multiple columns as well. It is possible for a table to have more than one column with unique values in the table, however only one primary key can be defined. Each row in a table must have a distinct value (or a set of values) in the column marked as the primary key. As a rule, primary key cannot contain a NULL value.

EEEE. Identifying Primary Key from Sample Requirements

As per the above-explained concept of the primary key, and the requirement scenario mentioned in the beginning of this lesson, the primary key can be identified in the following manner.

Entities & Attributes:

Entity	Attributes	Primary Key
Building	Building Name***(bname)***, Address, Phone Number	Building Name – bname
Apartment	Apartment ID***(aid)***, Covered Area, Status (available/rented), Rent	Apartment ID – aid
Customer	Customer ID***(cid)***, Name, Contact, CNIC	Customer ID – cid
Floor	***Floor Number***, Number of Apartment	Floor Number
Receipt	Receipt ID***(rid)***, Apartment Number, Date of Receipt, and Amount	Receipt ID – rid

As a rule, a primary key should be minimal and it should not contain unnecessary information. So in the above case, building name fits into the criteria or becoming a primary as rest of the attributes can be complex containing unnecessary information. Apartment ID is unique and the apartments in the building can have same covered area, rent or status making these attributes common. The primary key for rest of the entities is determined in the very same way.

FFFF.Super Key

A superkey is a set of attributes within a table whose values can be used to uniquely identify a tuple or row in a database. A super key can be single or combination of multiple columns/attributes which can uniquely identify value(s) between multiple rows. There can be multiple superkeys in a table but no two rows will share same combination or pair or super key. The definition of a superkey is a set of columns in a table for which there are no two rows that will share the same combination of values. So, the super key is unique for each and every row in the table. Primary key is name after concept of Super and sub-set, so a super key is essentially all the superset combinations of keys, which will of course uniquely identify a row in a table.

A minimal superkey is the minimum number of columns that can be used to uniquely identify a single row. In other words, the minimum number of columns, which when combined, will give a unique value for every row in the table.

GGGG. Identifying Super Key from Sample Requirement

As per the requirement scenario of buildings and apartments, consider the following as identification of Super Key.

- Entities:
  - Building (bname, address, phone#)
  - {(bname, address), (bname, phone)}
  - Minimal Sup

HHHH. Unique Key

A table in a database has unique key to identify a record or row within a table. In an entity-relationship diagram of a data model, one or more unique keys may be declared for each data entity. Each unique key is composed from one or more data attributes of that data entity. From the set of unique keys, a single unique key is selected and declared the primary key for that data entity. But unlike the primary keys, unique keys can have null values. Super keys, primary key can be declared as a type of unique keys.

IV. Candidate Key

A candidate key is the minimal number of attributes, whose value(s) uniquely identify each entity occurrence. The set of unique keys determined for a data entity is often referred to as the candidate keys for that data entity. A database table may have many candidate keys but at most one candidate key may be distinguished as the primary key . The candidate key is also termed as a potential primary key.

JJJJ. Identifying Candidate Key from Sample Requirements

Continuing with the same example of apartment rental for ease, the unique key and candidate keys can be identified from the scenario.

Entities & Attributes: (CK = Candidate Key & UK = Unique Key)

Entity	Attributes
Building	NTN# = **CK, UK**
Apartment	Apartment ID (aid) = **CK, UK)**
Customer	CNIC (UK) , Email (**UK)**

KKKK. Foreign Key

The foreign key is the field or attribute (or collection of fields) that is logically relate two table via common fields in other words, foreign key is used to create relationship between two tables. The primary key of the first table or parent table becomes the foreign key of the child table. When an attribute appears in more than one relation, its appearance usually represents a relationship between tuples of the two relations. Foreign key is not the primary key by default but it can serve the purpose of the primary key, but this is not a rule. Just like unique keys, foreign keys can have NULL values.

LLLL. Example of Foreign Key

As an example to illustrate foreign keys, consider the requirement scenario mentioned above.

An receipt database has a table and each receipt is associated with a particular customer. Customer details (such as name and address) are kept in a separate table; each customer is given a 'customerID' to identify it. Each receipt record has an attribute containing the customerID (cid) for that receipt. Then, the 'customerID' is the primary key in the customer table and that primary key will be the foreign key in the receipt table.

MMMM. Composite Primary Key (CPK)

Composite primary keys are the primary key which consists of more than one attributes. CPK combines more than one attribute of an entity to make a unique value. Sometimes it becomes necessary to combine more than one attribute in order to uniquely identify the records but as a rule, there will be just one primary key for each table. In relation between two entities, foreign key can become a part of CPK. CPK is unique as group and all the CPK values cannot be repeated. Besides, there are no composite foreign keys.

NNNN. Example of Composite Primary Key (CPK)

In our example of building & apartments renting scenario, a CPK can be created by joining two attributes of RECEIPT entity. For example, receipt ID and apartment number can be joined together to create a composite key which can uniquely identify each record. CPK are used where one attribute is not enough to uniquely identify records, CPK comes as a solution to this issue.

OOOO. Relationships

Relationship define how data is connected among the entities in a given system or in other words how one entity is logically connected with another entity of the system. Relationships in a database are said to be a combination of cardinality and optionality where optional relationship is one in which there may or may not be a matching record in parent / child table and cardinality represents the concept of "how many" and normally it is 0 or more. Equation for creating relationship is as follow:

$$Relationship\ (R) = Cardanilty\ (C) + Optionality\ (O)$$

The concepts of cardinality and optionality are explained below in details.

Relationships are bi-directional in nature; Relationship between two entities A and B is as follow:

i. Relationship from A to B
ii. Relationship from B to A

PPPP.Optionality with Example (…. Or ___)

Participation in entity relationship is either optional or mandatory (…. Or _____). This means ing entity occurrence in a particular relationship. & *Child* relationship, it is quite possible for parent not to have any child record. Therefore, child is optional to parents.

a child must have a parent and therefore, parent is mandatory to child. Hence minimum number of child record per parent in child table will show either 0 (…) or one

(_____) . A dotted or solid line shows this kind of relationship. To translate …… or _____ following rules are to be followed:

Optional: ------ zero or one

Mandatory: ____: Exactly one

QQQQ. Cardinality with Example (1 or >, <)

Cardinality expresses the minimum and maximum number of entity occurrences associated with one occurrence of the related entity. The cardinality tells what kind of relationship an entity has with another but it does not state whether it's mandatory or optional relationship. In the relational model, tables can be related as any of "one-to-many" or "many-to-many." This is said to be the cardinality of a given table in relation to another. Following rule are to be followed when translating cardinality

<or>: More than one

Cardinality is read with opposite entity.

RRRR. Basic Relationships

A cumbersome part of designing a relational database is dividing the data elements into related tables. Once you're ready to start working with the data, you rely on relationships between the tables to pull the data together in meaningful ways. The database system relies on matching values found in tables to form relationships. When a match is found, the system pulls the data from the tables to create a virtual record.

There are three kinds of relationships namely:

- One-to-Many
- One-to-One
- Many-to-Many

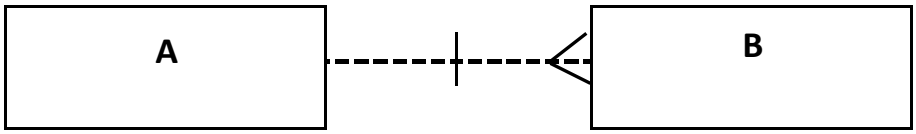
The first two types of the relationship have equivalent implementation in the database but the third type need conversion as it has no equal implementation in database. The details of these relationships are as follows.

a. One-To-Many Relationship

A one-to-many relationship is a type of cardinality that refers to the relationship between two entities A and B in which element of A may be linked to many elements of B, but a member of B is linked to only one element of A. One-to-many relationship is the most common type of relationship. In this type of relationship, a row in table A can have many matching rows in table B, but a row in table B can have only one matching row in table A. One-to-many relationship occurs when a parent record in one table can potentially reference several child records in another table.

As a rule, the Primary key of Parent table acts as Foreign Key in Child Table. Foreign Key is usually created as Composite Primary Key with PK of child table but this is no specific rule.

[illegible]ample-1 of One to [illegible]



Relationships are bi-directional:

A to B:

Optionality is Optional (zero or one) because line is dotted from A till half line and cardinality is more than one due to < symbol attach with B

Relationship:

Every instance of A is having one or more instances in B

B to A:

Optionality is optional due to dotted line from B till middle and cardinality is 1 because there is no > symbol with A

Relationship:

Every instance of B belong to exactly one A

Overall Relationship:

One to Many from A to B

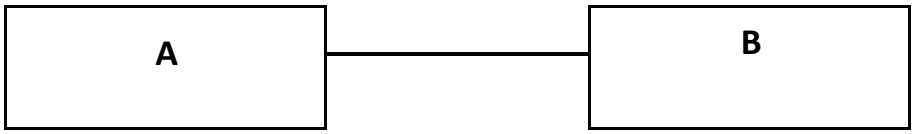
b. One-To-One Relationship

A one-to-one relationship is a type of cardinality that refers to the relationship between two entities A and B in which one element of A may only be linked to one element of B, and vice versa. As the one-to-one label implies, in this relationship, one entity can be related to only one other entity, and vice versa. It is important to note that a one-to-one relationship is not a property of the data, but rather of the relationship itself as there is no parent-child relationship in on-to-one relation scenario. The following are rules to implement One-to-One:

i. There will be foreign key in any one of the participating table.
ii. Foreign key will be made as Unique key.

c. Example of One-to-One Relationship

VU-Gateway



A to B:

Optionality is mandatory (exactly one) because line is solid from A till half line and cardinality is also one since there is no < or > symbol attach with B

Relationship:

Every instance of A is having exactly one instance in B

B to A:

Optionality is mandatory (exactly one) because line is solid from B till half line and cardinality is also one since there is no < or > symbol attach with A

Relationship:

Every instance of B is having exactly one instance in A

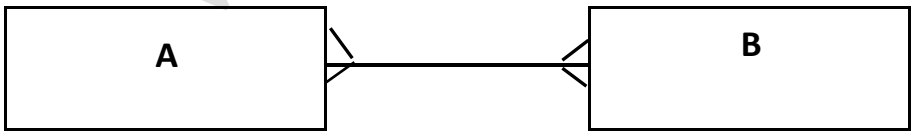
Overall Relationship:

One-to-One from A to B

d. Many-To-Many Relationship

A many-to-many relationship is a type of cardinality that refers to the relationship between two entities A and B in which A may contain a parent record for which there are many children in B and vice versa. This means that a parent row in one table contains several child rows in the second table, and vice versa. Many-to-Many relations are tricky to represent and are not supported directly in the relational environment. To represent this kind of relationships, a third entity or intersection table is created where PK of two table act as FK and CPK in third table.

e. Example of Many – to – Many Relationship



A to B:

Optionality is mandatory (exactly one) because line is solid from A till half line and cardinality is more than one due to < symbol attach with B

Relationship:

Every instance of A is having one or more instances in B

B to A:

Optionality is mandatory (exactly one) because line is solid from B till half line and cardinality is more than one due to < symbol attach with B

Relationship:

Every instance of A is having one or more instances in B

Overall Relationship:

Many- to- Many from A to B

TTTT. Referential Integrity Constraint

Referential integrity is a relational database concept in which multiple tables share a relationship based on the data stored in the tables, and that relationship must remain be consistent. The referential integrity rule is defined on a key (a column or set of columns) in one table that guarantees that the values in that key match the values in a key in a related table (the referenced value).In simple words, when a foreign key value is used it must reference a valid & existing primary key in the parent table otherwise the referential integrity will break. The child table can only be created if parent record is present as the child table is dependent on parent table. To create a child table, there must be a solid reference (parent table) as theorized by the referential integrity concept.

UUUU. Cascading

The purpose of the Foreign Key is to identify a particular row of the referenced table, it is required that the foreign key has a valid reference to from the referenced table or has NULL value. This rule is called the referential integrity constraint and violation of these constraints can create many issues. Cascading is the mechanisms to ensure the foreign keys have valid reference in the parent table. Cascading defines the behavior of foreign key when the record from the parent table is deleted or updated. To maintain referential and data integrity, cascading define two rules; CASCADE DELETE and CASCADE UPDATE. These are discussed in detail in the following lines.

VVVV. Cascade Delete

This rule states that if a row in the referenced table is deleted, then all rows in the referencing table with a foreign key value equal to the primary key value of the row should also be deleted. In simple words, if a parent row is deleted, then all the child records that are referencing to parent record will be deleted automatically. Consider the following example in order to understand the concept in a better way. The record can be deleted from the child and this will have no impact on the parent table.

Example:

Suppose company X has 2 tables, an Employee table, and a Department table. In the Employee table we have 2 columns – the employee ID and the Department ID. In the Employee Department table, we have 2 columns – the Department ID and the Department name for the given ID. Department ID is the foreign key in the Employee table. Now suppose we wanted to remove a department due to any reason, when a department is deleted, any rows in the employee table, that references the DepartmentID, are also deleted.

WWWW. Cascade Update

If the primary key value of a row in a referenced table is updated, all rows in the referencing table with a foreign key value equal to the primary key value of this row, should also be updated to the new value. As primary keys when migrated to another table to create logical relation becomes foreign key, so the value of foreign key must be dependent on primary key and hence it would be updated when the value of primary key is updated.

Example:

Continuing with the previous example of employee and department table, now suppose we wanted to update a department ID due to any reason, when a department ID is updated, any rows in the employee table, that references the DepartmentID, are also updated.

XXXX. Restrict Delete and Update

The concepts articulate that the parent record can neither be deleted nor updated if the child record exists. There is no restriction in deleting or updating the child records. The restrict Delete and Restrict Update prevents the deletion of data from the master table. In other words, restrict Delete or Update modifies the behavior of master table, not the child table.

YYYY. Surrogate Keys

Surrogate key is an artificial key which is not a part of the system. The purpose of the surrogate keys is to avoid the complexity of the composite primary keys. In order to enable surrogate, a single column is to be added in relevant table. Whenever the primary key is considered to be unsuitable, designers use surrogate keys which are artificially produced (most often system generated) and generally in numeric values.

ZZZZ. Example of Surrogate Keys

In our building-apartment rental scenario, the receipt ID can serve as an example of Surrogate Key. The receipt ID can be added automatically in a sequence. Instead of using a combination of one or more attributes of RECEIPT entity as a primary key, it would be more appropriate to use a sequential and auto generated ID as a primary key which can uniquely identify each record of RECEIPT.

VU Gateway

Module 10: Extended Entity Relationship Diagram

A. Extended Entity Relationship Diagram (EERD)

The EERD incorporate the extension to the original Entity Relationship Diagram (ERD) Model with reusability concept. It was developed to reflect more precisely the properties and constraints that are found in more complex systems. Common attributes which are repeated among multiple entities are placed in Super Type / Super entity and are shared with Sub-types to avoid redundancy. Attributes and relationships of super types are shared with sub-types but this relationship is not bottom-up meaning Super type can't access any attributes or relationship of subtypes.

The EER model includes all of the concepts introduced by the ER model. Additionally it includes the concepts of a Sub-Type and Super-Type. Extended ERDs are high-level conceptual models that accurately represent the requirements of complex databases.

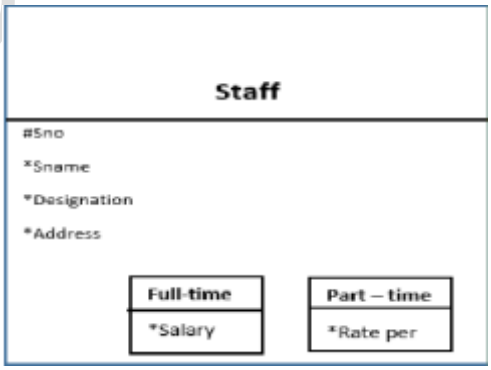
AAAAA. Super Types

Common attributes are placed in super type. It is also known as super class and many sub classes or subtypes can be derived from a super type. The details of the super types are available in the subtypes but on the other side, super types do not know anything about subtypes. In simple words, it can be defined as an entity type that has a relationship with one or more subtypes.

BBBBB. Sub Types

Sub Type or sub class is derived from the super class and it can have its own attributes and relationships. Besides having own relationships and attributes, the sub types inherit all the relationships and attributes of a super class. A class that is derived from another class is called a subclass (also a derived class). The class from which the subclass is derived is called a super class (also a base class or a parent class).

CCCCC. Example of Super Type & Sub Type

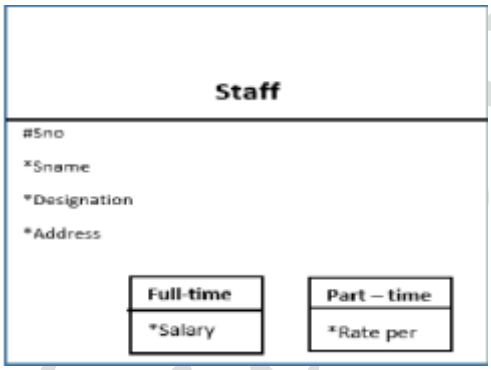


The diagram above explain the concept of super-type and sub-type. Common attributes (sno, sname, designation, address) are placed in Staff entity (Super type) as there will be multiple employee type (Full-time, part-time) and every employee type will share common attributes so instead of writing (sno, sname, designation, address) multiple times for Full-time and Part-time sub-types these 4 attributes are written only once and are shared by super type.

DDDDD.

It's the first property of Super & Sub Types which states that there should be at least two sub types and each subtype should have at least one attribute. Subtype without no specific attributes is not a valid Sub-type In other words, every instance of the super type is also an instance of one of the sub types.

EEEEE. Example of Exhaustive

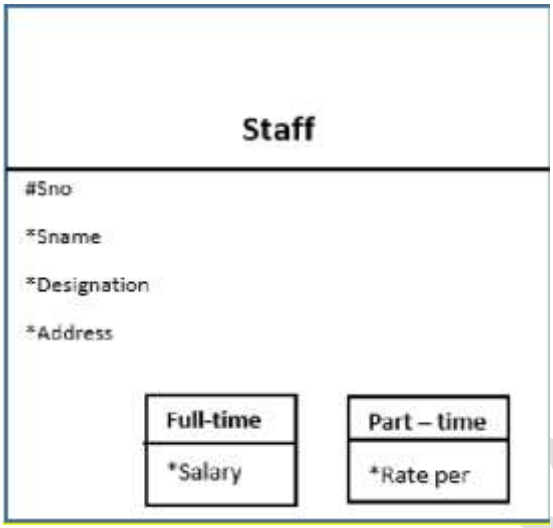


If we remove Salary attribute from sub-type Full-time then its not a valid sub-type because as per Exhaustive rule there has to be at least one specific attribute for sub-type to qualify to be a valid sub-type

F[illegible]F. Mutually Exclusive

[illegible] sub types are mutually exclusive as they don't know about each other. Attributes and relationship which a specific sub-type is having with other entities are not shareable or now known to other sub-types in the system. Every sub-type maintains isolation of relationships and attributes from other sub-types. This rule or property articulates that every instance of the super type is of one and only one subtype.

GGGGG. Example of Mutually Exclusive



In the diagram above, for example if there is sno=1 at any particular time, then this sno(1) can be associated with only one of its sub-type i-e either Full-time or Part-time; not the both sub-types. If sno=1 is associated with Full-time then sub-type part-time is not aware about this association due to mutually exclusive rule.

HHHHH. Example of EERD

Consider the following scenario:

In HR System there are different types of employees working in an organization mainly full-time, part-time. Only Full-time employee can avail the facility of loan however all type of employee can apply for study-leaves. Account department prepare the salary slip in the same way for all the employees.

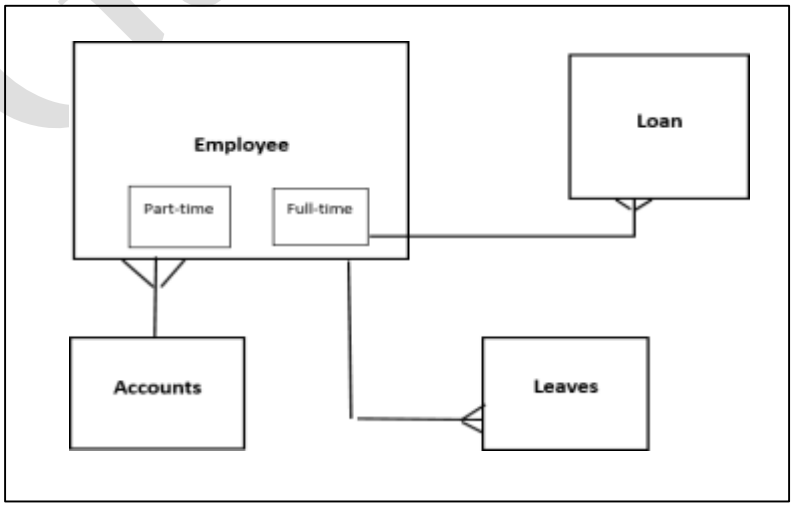


Figure 6: Example of EERD

VU Gateway

Module 11: Example of Entity Relationship Diagram (ERD)

A. Example 01: Scenario and Generating ERD

Recall the Building – Apartment Renting Scenario:

Scenario:

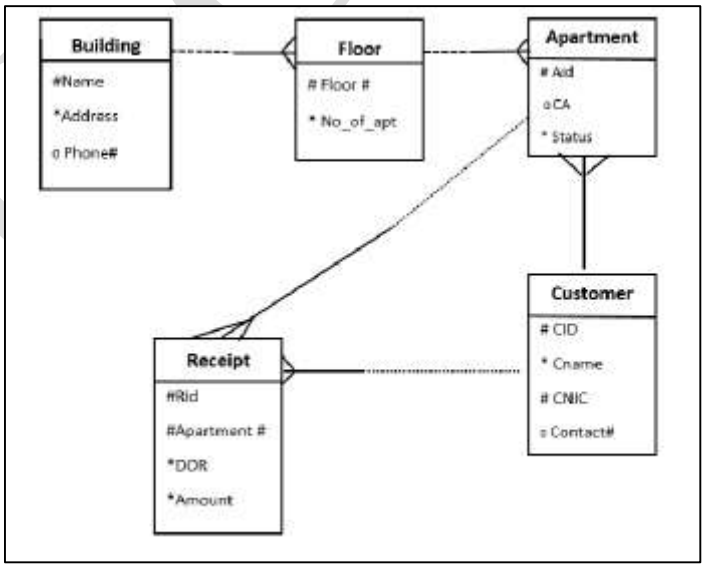
In a Building – Apartment renting scenario, there are Apartments, building and customer. There are multiple floors in the building and on each floor there are multiple apartments, floors can have zero or no apartment. Each apartment can be rented at most by one customer but customer can rent out multiple apartments from same building, apartment can be available on multiple floors or same. At the end of month a recipe is generated against which a rent is deposited

The entities and attributes can be derived in the following way:

Entities & Attributes:

Entity	Attributes
Building	Building Name (bname), Address, Phone Number
Apartment	Apartment ID (aid), Covered Area, Status (available/rented), Ren
Customer	Customer ID (cid), Name, Contact, CNIC
Floor	Floor Number, Number of Apartments
Receipt	Receipt ID (rid), Apartment Number, Date of Receipt, Amount

V. Generating ERD



JJJJJ. Generating Physical Data Model

In ERD it's not possible to show foreign keys as a rule but it is possible in Physical Data Model to show attributes and foreign keys.

As rule of thumb primary key of parent table is referred as foreign key in child table in One-to-Many relationship.

In Many-to-Many a new third table is introduced where primary key of both tables are written as FK and made part of CPK.

In One-to-One any one table will contain FK but FK will be made as UK as rule.

Physical Data model is last step towards before database Implementation.

KKKKK. Physical Data Model of ERD

- Building(bname, address)
- Floor (floor #,no_apt, bname)
  Assumption:
  Floor# can be repeated among multiple building due to th[illegible]ame i[illegible]ade[illegible]rt
  Composite primary key along with Floor # in floor table

- Customer (cid, name, address, contact#)
- Apartment(aid, status, floor#, bname, CA, cid)
- Receipt (rid, aid, cid,rent,dor)

B. Example 02: Scenario and Generating ERD

Consider the following scenario:

Scenario # 2:

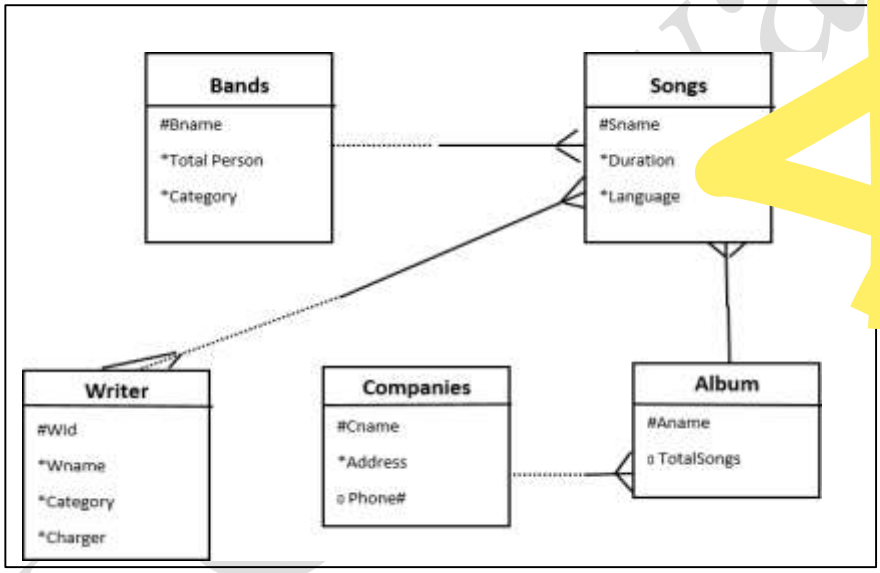
There are musical bands which record songs and request musical companies to launch their songs in the form of Album. Songs are written by song writers. Album can contain at least one song to be album and max of 12 songs. Writer can write multiple songs but songs are usually written irrespective of the number of person in the bands.

The entities and attributes can be derived in the following way:

Entities & Attributes:

Entity	Attributes
Bands	bname,total person,cat
Songs	sname, duration,language
Writer	wid, wname, charges, category
Company	cname, address, phone#
Album	aname, total_songs

LLLLL. Generating ERD of the Scenario



MMMMM. Physical Data Model of ERD:

- Band (bname,totalperson,cat)
- Company(cname, address, phone#)
- Songs(sname,duration,language,bname,aname)
- Writer(wid, wname, charges, category)
- Album(aname, total_songs,cname)
- Writer_song(wid,sname)

C. Example 03: Scenario and Generating ERD

Consider the following scenario:

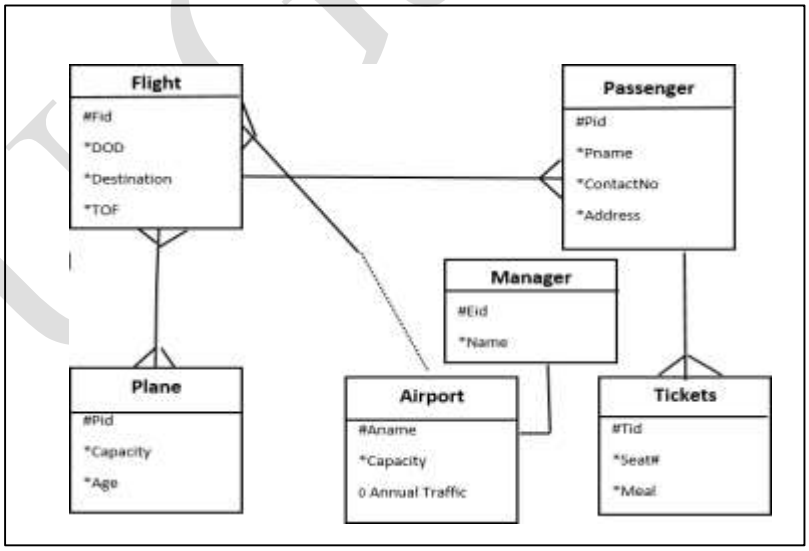
Scenario:

In airport management system, there are several flights which are either landing or taking off from Airport. There is more than one passenger on a particular flight. Passengers are allocated seat # when they purchase ticket. Plane can be associated to different flights during its lifecycle. Passenger can order different meals during flight and details are mentioned on the ticket in the form of either Asian or Non-Asian. Airport is managed by a manager.

Entities & Attributes:

Entity	Attributes
Airport	Aname, capacity, annual traffic
Manager	Employee ID (eid) ,Airport Name (aname),name
Flight	Flight ID (fid), Date of Departure (dod), Time of Dep. (tod) ,destination,aname
Passenger	Passenger ID (pid),pname,fid,contact,address,date
Ticket	Flight ID (fid), Ticket ID (tid), Passenger ID (pid),seat number,meal
Plane	Plane ID (pid),capacity,age

NNNNN. Generating ERD of the Scenario



OOOOO. Generating Physical Model from ERD

- Airport (aname, capacity, annual traffic)
- Manager (eid,aname,name)- **aname is FK and UK due to One-to-One relationship with Manager.**

- Flight (fid,dod,tod,destination,aname)
- Passenger(pid,pname,fid,contact,address,date)
- Ticket(fid,tid,pid,seat#,meal)
- Plane(pid,capacity,age)
- Plane_flight(pid,fid) – **Resolving Many-to-Many between flight and Plane**

VU-Gateway

VU-Gateway